1 Train a Small Model

Linear regression: y = 2x + 1

Trains a dense neural network on synthetic data. The true function is **y = 2x + 1**. After training the model learns these weights.

```
// Model architecture
const model = tf.sequential();
model.add(tf.layers.dense({
  units: 1,
  inputShape: [1]
}));
model.compile({
  optimizer: 'sgd',
  loss: 'meanSquaredError'
});
await model.fit(xs, ys, {
  epochs: 200
});
```

▶ Train Model

```
> Model architecture: Dense(1) |
optimizer: sgd | loss: MSE
> Training 200 epochs…
Epoch 50/200 — loss: 0.148111
Epoch 100/200 — loss: 0.019953
```

2 Save to LocalStorage

Persist model weights in browser

Serializes the trained model (architecture + weights) into browser LocalStorage using `tf.io.localstorage`.

```
// Save to LocalStorage
await model.save(
  'localstorage://co3-model'
);

// Keys stored:
// tensorflowjs_models/co3-model/info
// tensorflowjs_models/co3-model/model_topology
// tensorflowjs_models/co3-model/weight_specs
// tensorflowjs_models/co3-model/weight_data
```

💾 Save to LocalStorage

```
> Saving model to LocalStorage…
> Model saved! Keys written:
tensorflowjs_models/co3-
model/model_metadata
tensorflowjs_models/co3-model/info
tensorflowjs_models/co3-model/weight_data
tensorflowjs_models/co3-
```

3 Reload & Verify Predictions

Load from LocalStorage, compare outputs

Loads the saved model from LocalStorage, then compares predictions to the original trained model on the same inputs.

```
// Reload from LocalStorage
const loaded = await tf.loadLayersModel(
  'localstorage://co3-model'
);

// Predict and compare
const orig = model.predict(testX);
const reload = loaded.predict(testX);
```

🔄 Reload & Verify

```
> Model loaded successfully ✓
> Comparing predictions:
x=-2: orig=-3.0181, reload=-3.0181 ✓
x=-1: orig=-1.0184, reload=-1.0184 ✓
x=0: orig=0.9813, reload=0.9813 ✓
x=1: orig=2.981, reload=2.981 ✓
x=2: orig=4.9807, reload=4.9807 ✓
x=5: orig=10.9798, reload=10.9798 ✓
x=10: orig=20.9784, reload=20.9784 ✓
> All predictions match ✓
```

PREDICTION COMPARISON

Input x	Expected y=2x+1	Original	Reloaded	Match
[illegible]	[illegible]	[illegible]	[illegible]	✓

4 Export to Files & Re-Import

Download JSON + bin, then re-upload

Exports the model as `model.json` `weights.bin` files. Then re-import via file input and verify predictions again.

```
// Export as files (downloads 2 files)
await model.save('downloads://co3-model');

// Re-import from file input
const reimported = await
  tf.loadLayersModel(
    tf.io.browserFiles([
      jsonFile, weightsFile
    ])
  );
```

📤 Export Files

Re-import: select model.json then weights.bin

Choose file No file chosen

Choose file No file chosen

📥 Import & Verify

```
> Exporting model to files…
> Downloaded: co3-model.json
(architecture)
> Downloaded: co3-model.weights.bin
(weights)
> Now select these 2 files above to re-
import ↑
```